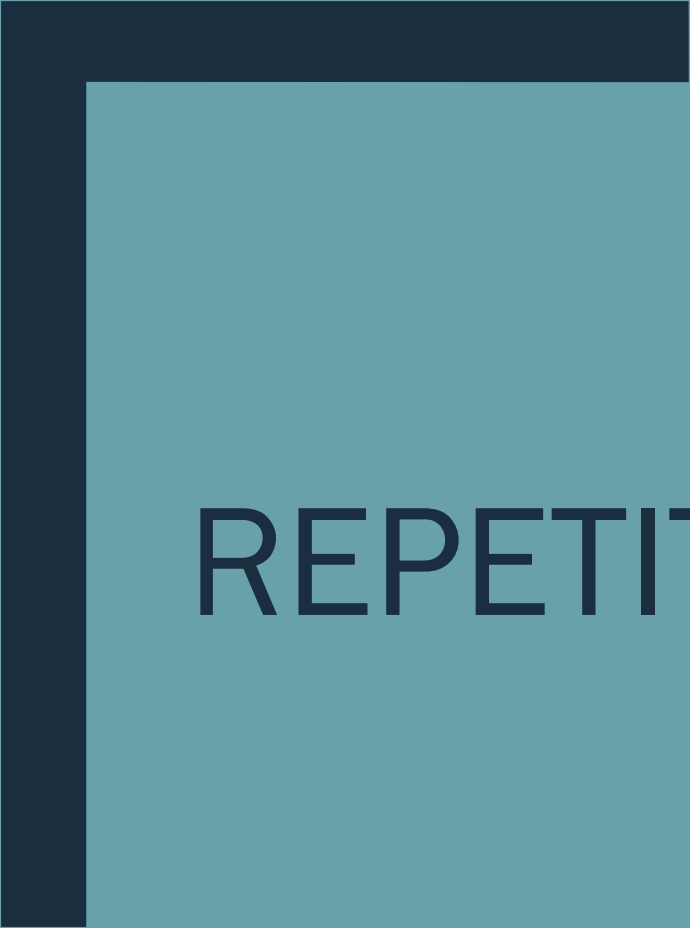




CEN111

REPETITION STRUCTURES I

Structures

- Sequence structures:
 - *Programs executed sequentially by default*
- Selection structures:
 - *if, if...else, and switch*
- Repetition structures:
 - *while, do...while and for*

Repetition in Programs

- loop
 - *a control structure that repeats a group of steps in a program*
- loop body
 - *the statements that are repeated in the loop*

Repetition in Programs

- Counter-controlled iteration

- *Counter-controlled iteration is sometimes called **definite iteration** because we know in advance exactly how many times the loop will be executed.*

- Sentinel-controlled iteration

- *Sentinel-controlled iteration is sometimes called **indefinite iteration** because it's not known in advance how many times the loop will be executed.*

Repetition in Programs

- In counter-controlled iteration, a control variable is used to count the number of iterations.
- The control variable is incremented (usually by 1) each time the group of instructions is performed.
- When the value of the control variable indicates that the correct number of iterations has been performed, the loop terminates and execution continues with the statement after the iteration statement.

Repetition in Programs

- Sentinel values are used to control iteration when:
 - *The precise number of iterations isn't known in advance, and*
 - *The loop includes statements that obtain data each time the loop is performed.*
- The sentinel value indicates “end of data.”
- The sentinel is entered after all regular data items have been supplied to the program.
- Sentinels must be distinct from regular data items.

Counting Loops

- Counter-controlled iteration requires:
 - *The name of a control variable (or loop counter).*
 - *The initial value of the control variable.*
 - *The increment (or decrement) by which the control variable is modified each time through the loop.*
 - *The condition that tests for the final value of the control variable (i.e., whether looping should continue).*

while Statement Syntax

```
while (loop repetition condition)
{
    statement;
}
```

while Statement Syntax



```
/* display N asterisks. */  
  
count_star = 0  
while (count_star < N)  
{  
    printf("*");  
    count_star = count_star + 1;  
}
```

General Conditional Loop

1. Initialize loop control variable.
2. As long as exit condition hasn't been met
 3. *Continue processing*



```
/* Counter-controlled iteration */
#include <stdio.h>
int main(void)
{
    int counter=1;           // initialization
    while (counter<=10)      // iteration condition
    {
        printf("%d\n",counter);
        counter++;           // increment
    }
    return 0;
}
```

Loop Control Components

- initialization of the loop control variable
 - test of the loop repetition condition
 - change (update) of the loop control variable
-
- the `for` loop supplies a designated place for each of these three components

The `for` Statement Syntax

`for` (initialization expression; loop repetition condition; update expression)
statement;



```
/* Display N asterisks. */  
for (count_star = 0; count_star < N; count_star += 1)  
    printf("*");
```



```
/* Counter-controlled iteration with the for statement*/
```

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
/* initialization, iteration condition and increment  
are all included in the for statement */
```

```
for (int counter=1;counter<=10;counter++)
```

```
{
```

```
    printf("%d\n",counter);
```

```
}
```

```
return 0;
```

```
}
```

for Iteration Statement

- When the `for` statement begins executing, the control variable `counter` is initialized to `1`.
- Then, the loop-continuation condition `counter <= 10` is checked.
- Because the initial value of `counter` is `1`, the condition is satisfied, so the `printf` statement prints the value of `counter`, namely `1`.
- The control variable `counter` is then incremented by the expression `counter++`, and the loop begins again with the loop-continuation test.

for Iteration Statement

- Because the control variable is now equal to 2, the final value is not exceeded, so the program performs the `printf` statement again.
- This process continues until the control variable counter is incremented to its final value of 11—this causes the loop-continuation test to fail, and iteration terminates.
- The program continues by performing the first statement after the `for` statement (in this case, the end of the program).

for Iteration Statement

- Notice that the for statement “does it all”—it specifies each of the items needed for counter-controlled iteration with a control variable.
- If there’s more than one statement in the body of the for, braces are required to define the body of the loop.

for Iteration Statement

Off-By-One Errors

- Notice that the loop-continuation condition `counter <= 10`.
- If you incorrectly wrote `counter < 10`, then the loop would be executed only 9 times.
- This is a common logic error called an **off-by-one error**.

for Iteration Statement

Off-By-One Errors

- Using the final value in the condition of a **while** or **for** statement and using the **<=** relational operator can help avoid off-by-one errors. For a loop used to print the values 1 to 10, for example, the loop continuation condition should be **counter <= 10** rather than **counter < 11** or **counter < 10**.

for Iteration Statement

Comma-Separated Lists of Expressions

- Often, the initialization and increment expressions are comma-separated lists of expressions.
- The commas as used here are actually comma operators that guarantee that lists of expressions evaluate from left to right.

for Iteration Statement

- The comma operator is most often used in the **for** statement.
- Its primary use is to enable you to use multiple initialization and/or multiple increment expressions.
- For example, there may be two control variables in a single for statement that must be initialized and incremented.

for Iteration Statement

Expressions in the for Statement's Header Are Optional

- The three expressions in the for statement are optional.
- If the condition expression is omitted, C assumes that the condition is true, thus creating an infinite loop.
- You may omit the initialization expression if the control variable is initialized elsewhere in the program.
- The increment may be omitted if it's calculated by statements in the body of the for statement or if no increment is needed.

for Iteration Statement

Increment Expression Acts Like a Standalone Statement

- The increment expression in the for statement acts like a stand-alone C statement at the end of the body of the for.

- Therefore, the expressions

```
counter = counter + 1  
counter += 1  
++counter  
counter++
```

are all equivalent in the increment part of the for statement.

- Because the variable being preincremented or postincremented here does not appear in a larger expression, both forms of incrementing have the same effect.
- The two semicolons in the for statement are required.

for Iteration Statement

- The initialization, loop-continuation condition and increment can contain arithmetic expressions. For example, if $x = 2$ and $y = 10$, the statement

`for (j = x; j <= 4 * x * y; j += y / x)`

is equivalent to the statement

`for (j = 2; j <= 80; j += 5)`

- The “increment” may be negative (in which case it’s really a decrement and the loop actually counts downward).
- If the loop-continuation condition is initially false, the loop body does not execute. Instead, execution proceeds with the statement following the for statement.

Examples Using the `for` Statement

- The following examples show methods of varying the control variable in a `for` statement.

- Vary the control variable from 1 to 100 in increments of 1.
`for (i = 1; i <= 100; i++)`
- Vary the control variable from 100 to 1 in increments of -1 (decrements of 1).
`for (i = 100; i >= 1; i--)`
- Vary the control variable from 7 to 77 in steps of 7.
`for (i = 7; i <= 77; i += 7)`
- Vary the control variable from 20 to 2 in steps of -2.
`for (i = 20; i >= 2; i -= 2)`
- Vary the control variable over the following sequence of values: 2, 5, 8, 11, 14, 17.
`for (j = 2; j <= 17; j += 3)`
- Vary the control variable over the following sequence of values: 44, 33, 22, 11, 0.
`for (j = 44; j >= 0; j -= 11)`

Examples Using the `for` Statement

- Application: Summing the Even Integers from 2 to 100

Homework

- Write a C program to find the factorial of an integer entered by user.
- Write a C program to count number of digits in an integer entered by user.